



Bug Hunter — Upgrade Guide

How to move your live Bug Hunter from the old version to the new one — the easy, safe way.

Version 2.6 → 3.0

★ Read this first (the 30-second version)

Upgrading uses the **same two steps you already know**: get the new code, then run the deploy script. **Your database is never touched**. All your bugs, users, projects, comments and files stay exactly where they are. This guide walks you through it slowly, and shows you how to undo it if you ever want to.

1. What changes, and what stays the same

✓ Stays EXACTLY the same

- All your data — bugs, requirements, tasks, projects, events, comments, attachments, history.
- Your login accounts and passwords.
- Your web address and port (`:8765`).
- Your database (not even scratched).
- The commands you run to deploy.

🔄 What's new in 3.0

- A faster, modern look (the old plain web pages become a smooth app).
- Each person can be limited to only their projects.
- In-app notifications & optional web push.
- Item links, bulk actions, richer reports, the "Sleuth" assistant.
- A new *show/hide password* eye button on every password box.

The big change under the hood is the website technology (from plain JavaScript to "React"). The good news: your **server doesn't need to know or care** — the new look is pre-packaged inside the code, so deploying it is no harder than before.

2. The upgrade — 6 simple steps

Do these on your server (the VM), logged in the way you normally do. Type each command and press Enter.

1 Go into the Bug Hunter folder

This is the folder that has `deploy.sh` inside it.

```
cd /path/to/bug-hunter
```

2 Make a safety copy of the database (takes a few seconds)

This saves your whole database into a single file, just in case. You will almost certainly never need it — but it costs nothing to have.

```
docker exec -t bugtracker_db pg_dump -U bugtracker bugtracker > db-backup-before-3.0.sql
```

If you changed the database name in your `.env`, use that name in place of `bugtracker`.

3 Get the new version 3.0

```
git pull
```

This downloads version 3.0. Your `.env` file and your `secrets/` folder are left alone.

No “git” on the server?

Instead, copy the new Bug Hunter folder onto the server and replace the old files — but **keep your existing `.env` file and `secrets/` folder** (don’t overwrite those two).

4 Run the deploy script

```
./deploy.sh
```

This builds the new version and restarts it. Wait until you see the green “**Bug Hunter deployed successfully!**” message. The script is written to **never touch your database** — it even prints a line confirming that.

5 Open it and log in

Visit `http://YOUR-SERVER-ADDRESS:8765` and log in with your usual admin account. Everything should be right where you left it.

6 Switch on access for your team (new — one time only)

Version 3.0 lets each teammate see **only the projects they belong to**. Right after the upgrade, managers and normal users will see an **empty screen**, because nobody is assigned to a project yet. This is expected.

To fix it (about 2 minutes): log in as **admin** → open **Users** → click a person → tick their **Projects** → **Save**. Repeat for each manager / user. **Admins always see everything**, so you’re never locked out.

3. Why your database is 100% safe

Your data lives in a sealed “vault” — a Docker storage volume named `bugtracker_pgdata`. Upgrading the *app* does not open, empty, or replace that vault. The app and the data are kept separate on purpose.

✓ Checked against your real 2.6 backup image

I compared your actual 2.6 production image with version 3.0. The upgrade:

- **Adds 6 new, empty tables** for the new features.
- **Adds 2 new columns** to existing tables — both with safe automatic values, so existing rows just fill in instantly.
- **Deletes nothing. Renames nothing. Changes no existing row.**

The new version creates those tables and columns by itself the first time it starts. Run it again and it simply notices everything already exists and moves on.

✗ The one command to never run during an upgrade

Only this erases data, and it forces you to type `YES` first:

```
./down.sh --wipe-db      ← deletes the database. DO NOT use.
```

Plain `./down.sh` (no extra words) is completely safe — it just stops the app and keeps all your data.

4. One quick check before you start — only if you use “https”

If your Bug Hunter shows a **padlock** in the browser (https), open your `.env` file and make sure this line has a long random value:

```
SESSION_SECRET=...a long random string...
```

If it's empty, the new version won't start over https. Create a value by running this and pasting the result after `SESSION_SECRET=`:

```
openssl rand -hex 32
```

If your current 2.6 already runs fine over https, this is almost certainly already set — just confirm it's there. If you use plain `http` on an internal network, you can skip this step.

5. If you ever want to undo it (rollback)

Don't worry — nothing is lost

You have **two** safety nets: the database backup file from Step 2, and your saved 2.6 image (`bugtracker_app_2.6.tar.gz`). Your data is safe either way.

A) Stop the new version (keeps all data):

```
./down.sh
```

B) Load your saved 2.6 image back into Docker:

```
docker load -i /path/to/bugtracker_app_2.6.tar.gz
```

Copy that backup file onto the server first if it's only on your PC.

C) Tell Docker to use the old image, then start it:

In `docker-compose.yml`, under the `app:` section, set `image: bugtracker_app:2.6` and put a `#` in front of the three `build:` lines. Then run `./deploy.sh`. (The old 2.6 image is often still on the server anyway, so this usually “just works”).

If you ever need to restore the data itself from the Step-2 backup:

```
cat db-backup-before-3.0.sql | docker exec -i bugtracker_db psql -U bugtracker bugtracker
```

Not sure about any of this? Stop and ask your developer — your data is intact, so there's no rush.

6. A note for whoever edits the code (the developer)

The new app is built with React, but **the server does not build anything** — it just runs the already-built files saved inside the project (in `app/static`). That's what keeps the server side as simple as before. So before pushing new code to GitHub:

1. In the `frontend` folder, run `npm install` (first time only), then `npm run build`.
2. Commit everything, **including the updated `app/static` folder**.
3. Push to GitHub.

After that, the server's `git pull` + `./deploy.sh` works exactly like it always did — no Node, no build tools, nothing extra needed on the server.

Quick reference — the only commands you need

```
cd /path/to/bug-hunter                                # step 1
docker exec -t bugtracker_db pg_dump -U bugtracker bugtracker > db-backup-before-
3.0.sql # step 2
git pull                                                # step 3
./deploy.sh                                           # step 4
# then open http://YOUR-SERVER:8765 and assign projects to your team
```

Bug Hunter Upgrade Guide · Version 2.6 → 3.0 · Your database is never modified by these steps — only new, empty tables and columns are added automatically on first start.